



Nhóm 07

Work Sheet 0

Mục tiêu sản phẩm

- Giảm thời gian tra cứu SOP Standard Operating Procedure/Manual
- Giảm lỗi vận hành do human error
- Chuẩn hoá tri thức bảo trì knowledge base

User chính

- Kỹ thuật viên bảo trì
- Quản đốc
- Team vận hành

Insight

Constraint thực tế:

- Không có internet, hệ thống legacy, mạng yếu
 - Không thể dùng cloud đơn giản
 - Phải thiết kế on-prem/hybrid

Cost lớn và khó kiểm soát nếu làm sai

- Không chỉ là tiền model mà còn:
 - Compute, storage, integration
- Dùng sai kiến trúc → cost tăng nhanh khó scale

Trade-off rõ ràng giữa hiệu năng performance-reliability-cost

- Cloud: dễ nhưng phụ thuộc vào mạng, đắt

- On-prem: ổn định nhưng tốn setup nhưng chỉ lúc đầu

Work Sheet 1

Bối cảnh

- Nhà máy muốn AI assistant hỗ trợ kỹ thuật viên tra cứu SOP, log bảo trì, lỗi máy móc và checklist an toàn

Dữ liệu liên quan

- SOP, manual máy móc, log bảo trì, lịch vận hành

Mức độ nhạy cảm của dữ liệu

- Có nhạy cảm, dữ liệu chỉ được lưu hành nội bộ

Ràng buộc

- Nhiều hệ thống cũ
- Khu vực mạng nội bộ, thiết bị và mạng có thể không đồng đều
- Cần độ ổn định cao

Lựa chọn triển khai

Cloud

- Các vấn đề:
 - Không có internet
 - Latency không ổn định
 - Data nội bộ → rủi ro bảo mật
- Kết luận: cloud only không phù hợp constraint đề bài

On-prem

- Ưu điểm:
 - Chạy được offline

- Latency thấp
- Kiểm soát data
- Các vấn đề:
 - Model local bị giới hạn
 - Khó scale khi cần task phức tạp
 - Khoảng đầu tư ban đầu cao
- Kết luận: chạy được nhưng không linh hoạt, dễ bị giới bởi model AI local

Hybrid

- Local (on-prem/edge):
 - Query SOP, manual (RAG cơ bản)
 - Checklist, rule based
 - Log đơn giản
 - Khi không có mạng
- Cloud xử lý
 - Phân tích phức tạp
 - Training/update model
 - Phân tích tổng hợp

Kết luận: lựa chọn hybrid

- Ưu tiên:
 - Local → nhanh, ổn định
 - Cloud → mạnh, thông minh
- Fallback:
 - Nếu mất mạng → vẫn chạy local
- Lý do lựa chọn:
 - Mạng yếu → chạy local
 - Latency → thấp
 - Cost → tối ưu

- AI capality → vẫn dùng cloud khi cần
- Reliability → có fallback

Work Sheet 2

Ước lượng usage

- 50 users/ngày
- ~ 500 request/ngày
- Peak: 80-100 request/giờ
- 70%-80% request xử lý, 20-30% lên cloud

Ước lượng token

- ~120 request/ngày dùng cho cloud
- ~1100 tokens/request

Các lớp cost

Local

- Compute (server/edge device)
- Điện + vận hành
- Vector DB (RAG)
- Storage (SOP, log)

Cloud

- LLM API (task phức tạp)
- Backup/sync

Khác

- Logging & monitoring
- Human review
- Maintenance (update SOP, model)

Cost MVP

Local

- Server mini/GPU nhẹ: 300-800 usd (one-time)
- Vận hành: ~50-100 usd/tháng

Cloud

- ~4M tokens/tháng → ~40-100 usd/tháng

Khác

- Storage + logging: 50-100 usd
- Maintenance: 100-200 usd

Tổng

- ~250 - 500 usd/tháng (+ chi phí server ban đầu) → rẻ, ổn định hơn cloud only

Scale 5-10x

- Local compute tăng (có thể cần thêm node)
- Cloud cost tăng nhưng không đáng kể
- Logging + storage tăng đáng kể
- Phần tăng mạnh nhất
 - Local compute + system scaling phần còn chỉ là token

Cost driver lớn nhất của hệ thống

- Compute (LLM inference + embedding)
 - Vì sao:
 - Query diễn ra liên tục
 - Mỗi request (gồm embedding RAG, LLM generate)
 - Dùng cloud API → tính theo token → scale theo usage
 - Log + SOP thường dài → token input lớn → cost tăng

Hidden cost

- Integration + data processing (ETL)
 - Làm sạch SOP, manual (pdf xấu, scan)
 - OCR tài liệu
 - Chuẩn hoá log từ hệ thống legacy
 - Mapping dữ liệu giữa các hệ thống
- Vì sao nguy hiểm: không thấy ngay lúc demo, chiếm nhiều thời gian và nhân lực

Ước tính lạc quan

- Tỷ lệ request phản lên cloud + effort maintain
- Thực tế:
 - Nhiều case phức tạp hơn dự đoán
 - Data không sạch, phải xử lý lại

Work Sheet 3

3 chiến lược

- Model routing (local ↔ cloud)
- Semantic caching
- Selective interface (phân tầng request)

Model routing

- Tiết kiệm:
 - Giảm LLM cloud token cost
 - Tận dụng local model cho task đơn giản
- Lợi ích:
 - Giữ latency thấp
 - Giảm phụ thuộc cloud

- Phù hợp kiến trúc hybrid
- Trade-off:
 - Cần logic routing (phức tạp hơn)
 - Có thể routing sai → giảm chất lượng
- Thời điểm áp dụng: ngay từ MVP (vì đây là core của hybrid)

Semantic caching

- Tiết kiệm:
 - Giảm request lặp lại → giảm token + compute
 - Đặc biệt hiệu quả với SOP query
- Lợi ích:
 - Response nhanh hơn
 - Giảm tải hệ thống
- Trade-off:
 - Cache sai → trả lời lỗi thời
 - Cần cơ chế invalidate cache
- Thời điểm áp dụng: Ngay sau MVP (Phase 1) (khi thấy pattern query lặp lại)

Selective interface

- Tiết kiệm:
 - Chỉ dùng model mạnh khi cần → giảm cost inference tổng thể
- Lợi ích:
 - Tối ưu cost + performance
 - Tránh "overkill AI"
- Trade-off:
 - Cần classify request (logic thêm)
 - Có thể phân loại sai
- Thời điểm áp dụng:

- Sau khi có data usage thực tế (Phase 2)

Chốt chiến lược

- Làm ngay (MVP / Phase 1)
- Model Routing

Làm sau (Phase 2)

- Selective Inference (khi có data thật)

Work Sheet 4

Tình huống, tác động → user

- Traffic tăng đột biến
 - Tác động: chậm/timeout, queue dài, kỹ thuật viên không tra cứu kịp → ảnh hưởng vận hành
- Provider (cloud LLM) timeout
 - Tác động: các task phức tạp fail, user mất niềm tin nếu không có fallback
- Response chậm (latency cao)
 - Tác động: trải nghiệm kém, user bỏ dùng → quay lại cách thủ công

Phản ứng ngắn hạn & giải pháp dài hạn

Traffic spike

- Ngắn hạn: bật queue + rate limit, ưu tiên request real-time
- Dài hạn: scale local node (horizontal), tối ưu caching + routing

Provider timeout

- Ngắn hạn: retry (có backoff), fallback sang local model
- Dài hạn: multi-provider (không phụ thuộc 1 bên), giảm dependency cloud (tăng local capability)

Response chậm

- Ngắn hạn: trả partial answer + top docs, chuyển sang async nếu không critical
- Dài hạn: tối ưu RAG (chunk, index), dùng semantic caching

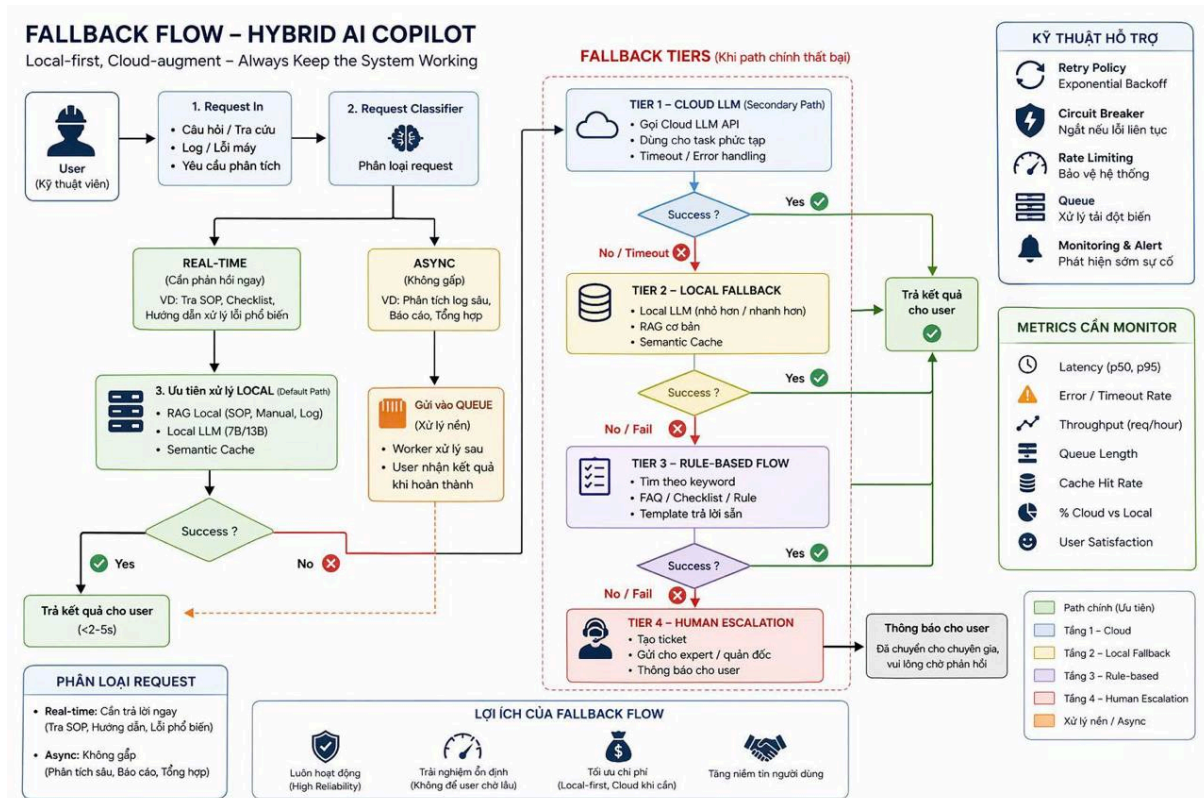
Metric cần monitor

- Latency (p50, p95)
- Error rate / timeout rate
- Throughput (request/giờ)
- Queue length
- Cache hit rate
- % request dùng cloud vs local
- User satisfaction (feedback)

Fallback proposal (quan trọng)

- Nguyên tắc: Không để hệ thống "chết hoàn toàn"
- Fallback tầng 1 (ưu tiên)
 - Local model (RAG cơ bản)
 - Semantic cache
- Fallback tầng 2
 - Rule-based: checklist, keyword search SOP
- Fallback cuối
 - Human escalation: gửi ticket / gọi expert
 - Kỹ thuật cần có:
 - Queue (xử lý burst)
 - Circuit breaker (ngắt khi provider lỗi)
 - Retry policy (exponential backoff)
- Phân loại request
 - Real-time: tra SOP, lỗi máy → xử lý ngay

- Async: phân tích log sâu → xử lý sau



Work Sheet 5

Self-rating (ví dụ)

Thành viên	Business/Product	Infra/Data/Ops	AI Engineer/Application
Lê Kim Dũng	3	2	3
Nguyễn Gia Bào	3	2	4
Nguyễn Dương Ninh	3	3	3

Điểm mạnh nhóm

- AI Engineering + System thinking (trung bình-khá)
- Infra chưa mạnh → cần bổ sung

Chọn Track Phase 2

- AI System / Applied AI (production-oriented)
- Focus:
 - RAG optimization
 - Hybrid deployment
 - Reliability & scaling

Kỹ năng cần bù

- Infra / System Design
 - queue, load balancing
 - distributed system cơ bản
- Data pipeline
 - xử lý log
 - ETL + cleaning
- AI system optimization
 - RAG tuning
 - caching, routing